

Type Systems

CS-3180

Language Design

- Syntax
- Paradigm
- Level
- One of the most important:



Language Design

- Syntax
- Paradigm
- Level
- One of the most important:
- Type System



Type System

- Often discuss data types
- Let's get definitions
- Match our intuition
- "Primitive" and "Complex" types

Type System

- Often discuss data types
- Let's get definitions
- Match our intuition
- "Primitive" and "Complex" types

An attribute or classification that tells the executor how the programmer intends to use a specific piece of data

Data Types

- Definition hints at some arbitrariness here
- How we "intend to use" data
- We are giving meaning to 1's and 0's

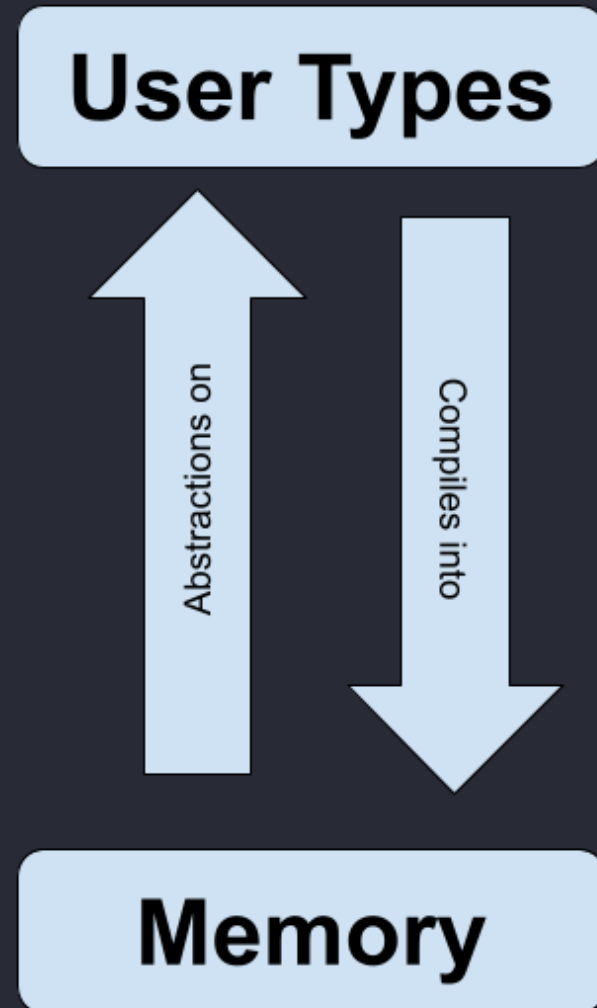


Data Types

- The lower level you are
- The less meaning a type has
- CPU instructions don't know what an `Animal` is
- ASM + C usually just use integers
- An abstraction on 1's and 0's

Data Types

- Encourage thinking about types from:
- Bottom up -> Abstractions on binary
- Over top down -> Compile into binary



Type System

- How a language handles types is one of the primary ways to differentiate them
- Aside from syntax of course
- How did we get to all the languages doing this differently?

C's Types

- C has a few useful types:
- `int`: Integer
- `char`: Character
- `float`: Floating point numbers
- `true/false`: Booleans via `stdbool.h`

Let's use them in
code

C's Types

- But does it really?:

C's Types

- But does it really?:
- ``int``: $\Rightarrow$ Integers

C's Types

- But does it really?:
- ``int``: $\Rightarrow$ Integers
- ``char``: $\Rightarrow$ Integers in a trenchcoat

C's Types

- But does it really?:
- ``int``: => Integers
- ``char``: => Integers in a trenchcoat
- ``floats``: => Stored as 32-bit ints

C's Types

- But does it really?:
- ``int``: => Integers
- ``char``: => Integers in a trenchcoat
- ``floats``: => Stored as 32-bit ints
- ``true/false``: => 1/0

C's Types

- But does it really?:
- ``int``: => Integers
- ``char``: => Integers in a trenchcoat
- ``floats``: => Stored as 32-bit ints
- ``true/false``: => 1/0
- `Pointers*` => Address = Integer

C's Types

- But does it really?:
- ``int``: => Integers
- ``char``: => Integers in a trenchcoat
- ``floats``: => Stored as 32-bit ints
- ``true/false``: => 1/0
- `Pointers*` => Address = Integer
- `Arrays` => `Pointers` => Integer

C is lying

- C treats basically every type as int under the hood
- Abstraction on binary again
- You still need to EXPLICITLY label types
- FORTRAN was one of the first to do this*



Type Coercion

- Because everything is an int
- C will seamlessly convert between types
- Implicitly convert between your types
- We'll call this behavior "Type Coercion"

Type Coercion

```
#include <stdio.h>

int main() {
    int a = 10;
    // ascii 65
    char b = 'A';
    float c = 5.5;

    float result = a + b + c;

    printf("%f", result);
    return 0;
}
```

Better Example

```
#include <stdio.h>

// expect int
void printNumber(int num) {
    printf("number was: %d\n", num);
}

int main() {
    double myPi = 3.14159;

    // c converts double to int
    printNumber(myPi);

    return 0;
}
```

- Let's try this in Java

Weak Typing

- This property of converting between types
- A weakly-typed system
- Note: The first example actually works in Java
- Strings are usually an edge case

Strong Typing

- If instead, a language **REQUIRES** that your types match
- We will call that a strongly typed language
- Java is an example of a strongly typed language



Typing Strength

- Type strength is always a spectrum
- A strong type system usually:
- EXPLICITLY cast between types
- Prevent operations on invalid types
- Minimize silent conversions

Examples of both:

- Weakly Typed:

- C
- Perl
- Javascript
- Bash

- Strongly Typed:

- Java
- Rust
- Go

Examples of both:

- Weakly Typed:

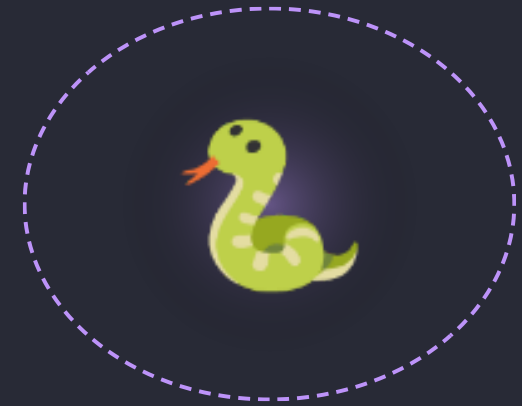
- C
- Perl
- Javascript
- Bash

- Strongly Typed:

- Java
- Rust
- Go
- Python?

Python?

- How is python a *strongly* typed language?
- We saw before how fast-and-loose it plays
- Let's look at a non-string example



```
def calculate_total(price, quantity):  
    return price * quantity  
  
print(calculate_total(4.99, 3))  
  
print(calculate_total(4.99, "three"))  
  
# you can actually catch these errors  
# introduce try/except  
try:  
    print(calculate_total(4.99, "three"))  
except TypeError as e:  
    print(f"Oopsies: {e}")
```

What's different?

- Well, let's do it in Java and see
- Type checking still occurs in both
- Difference: WHEN the type check occurs



Dynamic Typing

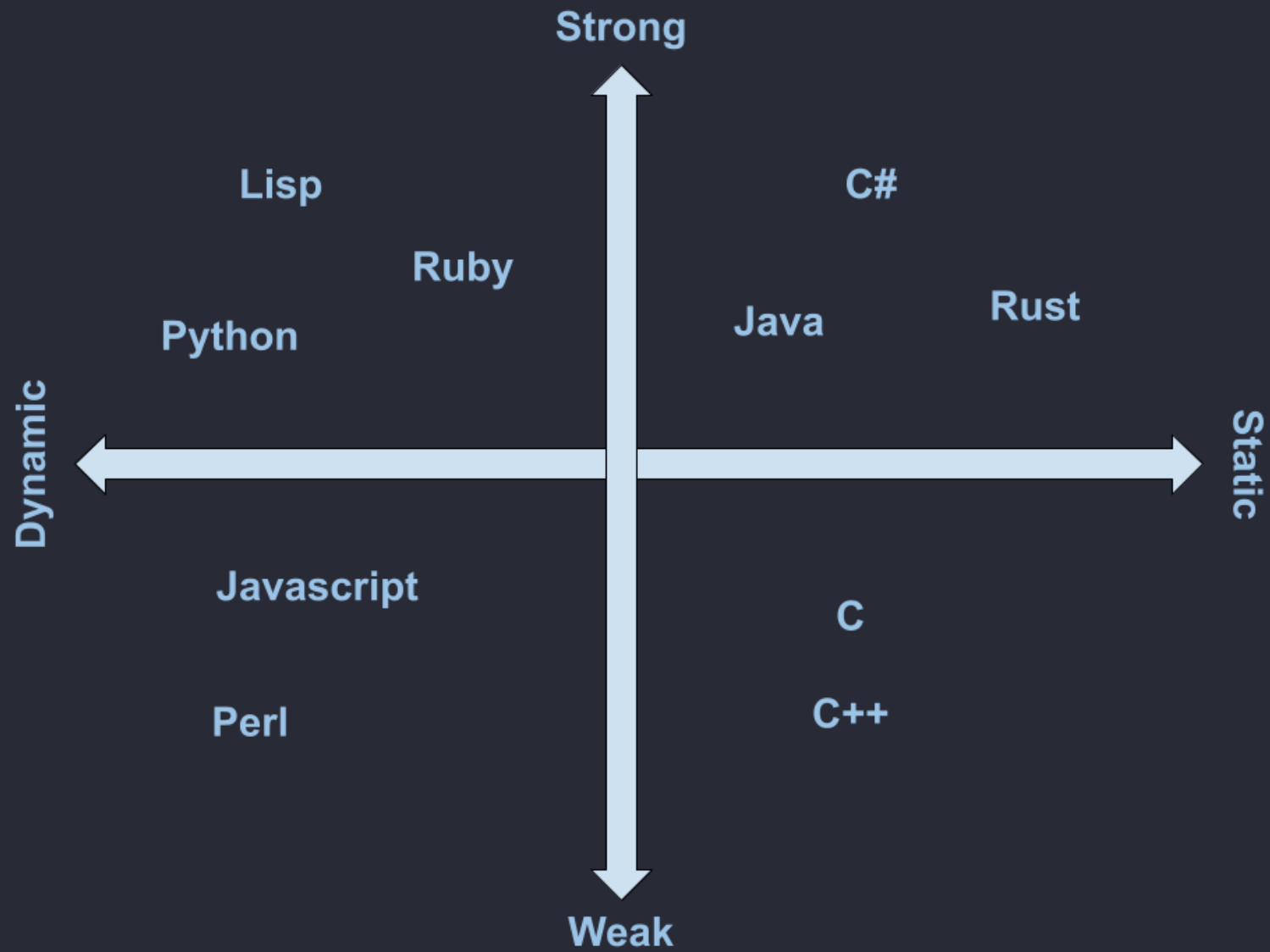
- We will call this property of waiting until RUN-TIME to check your types
- Dynamic typing
- Types are associated with the data, not the variable

Static Typing

- Opposite of this: static typing
- Variable types are checked prior to RUN-TIME
- Usually, this means they are checked at COMPILE-TIME
- Types are bound to the variable itself
- Also a spectrum

Typing

- Strong/Weak + Static/Dynamic types
- Are compatible with one another
- Often these are organized along a two-axis plot
- Let's look at one



Wrap Up

- Two ways of categorizing type systems
- Explicitness of Coercion (strong/weak)
- Time when types are checked (static/dynamic)
- Most languages fall somewhere in between
- Next time: Nuances

Questions?